

JavaScript

Antoine HINTZY
antoine.hintzy@ece.fr

Adapté par Laurent Debize

JS

Origines du Javascript



Mai 1995

Brendan Eich crée le langage Javascript en 10 jours pour le compte de la **Netscape Communications Corporation** afin de manipuler de façon simple des objets (informatique) fournis par une application hôte.

Mars 1996

Netscape intègre Javascript dans son navigateur web "**Netscape Navigator 2.0**".

Juin 1997

Javascript est standardisé sous le nom d'**ECMAScript** par Ecma International dans le standard **ECMA-262**.

Javascript est aujourd'hui une implémentation d'ECMAScript, soutenue par la fondation Mozilla.



Le langage Javascript



Langage de programmation de **scripts** créé pour le web.

Non compilé, un **interpréteur** (par exemple **V8** pour Chrome) se charge de le lire et de l'interpréter (réaliser les actions demandées).



Haut niveau (= loin du système)

Mais il permet de manipuler les fonctionnalités du système normalement réservées aux langages de bas niveau grâce à des APIs.

Le langage Javascript



Impératif

Description d'opérations en séquences d'instructions exécutées par l'ordinateur afin de modifier l'**état** du programme.

Fonctionnel

Imbrication de fonctions qui agissent comme des boîtes noires.
Possibilité d'utiliser des librairies comme *Lodash* ou *Ramda*.

Typage dynamique faible

Le typage est réalisé à la volée par la machine, lors de l'exécution du script.

ECMAScript

C'est la standardisation de Javascript.

Terme très employé de nos jours pour parler de Javascript.

Le *versioning* de Javascript a été abandonné, on se fie maintenant aux versions d'ECMAScript pour dire de quelle version de Javascript on parle.

Chaque navigateur implémente les nouvelles spécifications d'ECMAScript à son rythme.

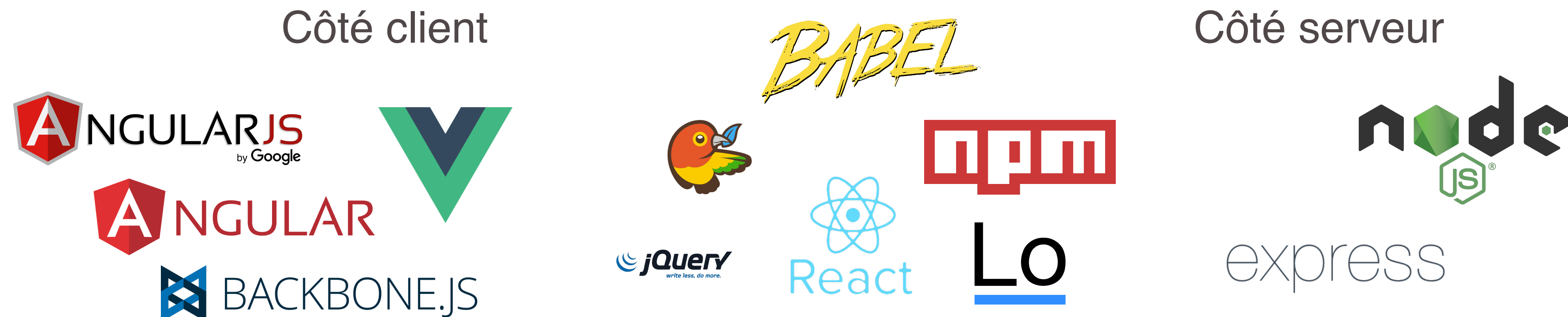
Le Javascript aujourd'hui

Le langage du Web. Créé pour le web.

Membre des 3 langages indispensables en web : HTML / CSS / Javascript

Un seul langage, côté client et côté serveur (avec **Node.js**) -> Développeurs **Full-Stack**

Apparition des **Single Page Applications** (site web sur une seule page HTML)



A yellow square containing the letters 'JS' in a bold, black, sans-serif font.

JS

Le langage Javascript

Documentation



<https://developer.mozilla.org/fr/docs/Web/JavaScript>

Exécuter du Javascript

✓ Côté serveur, directement avec Node.js <https://nodejs.org/en/download/>

✓ Côté client, dans un navigateur

- Directement dans une balise `<script>` dans un fichier HTML (à éviter)
`<script type="text/javascript"> /* Javascript */ </script>`
- Dans un fichier Javascript (.js), appelé par un fichier HTML (soit à la fin de `<head>` soit à la fin de `<body>`)
`<script type="text/javascript" src="./script.js"></script>`
- Outils de débogage du navigateur web (onglet console)

✓ Depuis des outils en ligne <https://jsfiddle.net/>

Hello World

```
console.log("Hello World");
```

- ▶ Affiche "Hello World" dans la console du navigateur

Dans un navigateur :

```
window.alert("Hello World");
```



```
alert("Hello World");
```

- ▶ Ouvre une pop-up d'alerte affichant "Hello World" (window est facultatif)

Variables

```
var nomVariable = "Valeur";  
// pas de typage explicite
```

- 3** instructions de déclaration de variable à partir d'**ES6** :
- var (toutes versions de JS/ES)
 - let
 - const

ES6⁺
uniquement

```
var maVariable = 3;  
maVariable = "Du texte";
```

Type : string ➤ "Du texte"

Il est possible de changer le type d'une variable à tout moment.

Le type des variables est déterminé lors de l'exécution du script.

Les déclarations de variables sont auto-remontées (on n'est pas obligé de déclarer les variables en haut du programme).

Déclaration de variables : var (let et const)

var Permet de déclarer une variable, en fournissant éventuellement une valeur pour l'initialiser.
Portée : fonction courante, sinon, contexte global

ES6⁺
uniquement

let Permet de déclarer une variable locale dans la portée d'un bloc (paire d'accolades) et éventuellement d'initialiser sa valeur.

const Déclare une constante en lecture seule.
Portée identique à let.

Variables : var (let et const)

```
function varTest() {  
  var x = 31;  
  if (true) {  
    var x = 71;    // c'est la même variable (dans la fonction varTest())  
    console.log(x); // 71  
  }  
  console.log(x);  // 71  
}
```

⁺
ES6

```
function letTest() {  
  let x = 31;  
  if (true) {  
    let x = 71;    // c'est une variable différente (dans le bloc du if)  
    console.log(x); // 71  
  }  
  console.log(x);  // 31  
}
```

Blocs en Javascript

Attention, il ne faut pas confondre les objets Javascript : { key1: valeur1, key2: valeur2 } avec les blocs.

Un bloc est délimité par une paire d'accolades, et influe sur la portée des variables, ils contiennent du code :

```
let a = 1;
{ // paire d'accolades pour un nouveau bloc
  let a = 2;
  console.log(a); // 2
} // le a (=2) n'existe plus en dehors du bloc !
console.log(a); // 1 !!
```

Les blocs sont utilisés pour : les boucles `for`, les `if`, `else`, `else if`, les `function`, ...
Mais peuvent également être utilisés de façon indépendante, tel que ci-dessus.

Types

Le type d'une variable Javascript n'est pas déclaré explicitement, il est défini par le moteur d'exécution JavaScript (= typage dynamique).

Pour connaître le type d'une variable, on peut utiliser l'opérateur **typeof**

Types de données

```
typeof 2.4
```

➤ number

```
typeof new String("24")
```

➤ object

```
typeof "24"
```

➤ string

```
typeof true
```

➤ boolean

```
typeof null
```

➤ object

```
typeof undefined
```

➤ undefined

Objets

En Javascript, on crée un objet entre une paire d'accolades :

```
var user = {  
  name: "Toto",  
  direBonjour: function () {  
    console.log("Bonjour !");  
  }  
};
```

Contrairement au JSON, on n'est pas obligé de mettre des accolades autour des clés.

Objets

```
var person1 = { // Création d'un objet Javascript entre accolades
  firstname: "Toto",
  lastname: "Tutu",
  sayHello: function () {
    console.log(`Hello, my name is ${this.firstname}!`);
  }
};

person1.age = 20; // L'objet person1 possède maintenant un attribut âge
console.log(person1.age); // Affiche 20

var first = "firstname";
console.log(person1[first]) // Affiche "Toto", identique à : person1.firstname
                           // et person1["firstname"]

person1.sayHello(); // Affiche "Hello, my name is Toto!"
delete(person1.age); // person1 n'a plus d'âge. person1.age === undefined
```

Bonne pratique

Utilisez le **débuggeur** de votre navigateur en ajoutant des points d'arrêt.
Ils vous permettront de visualiser l'état du programme à l'instant t (valeur des variables disponibles).

A yellow square containing the letters 'JS' in a bold, black, sans-serif font.

JS

Manipulation du DOM

Affichage d'une page web par le navigateur

Lorsque l'on se rend sur une page web, le navigateur parcourt les fichiers la composant (HTML, CSS, Javascript) et crée une **représentation (objet) de la structure de cette page web, appelée DOM**.

Le visuel de cette page web est ensuite généré par le navigateur, d'après le DOM qu'il a créé.

Le DOM est accessible et modifiable en Javascript, ce qui permet de modifier l'apparence de la page sans la recharger.

DOM (Document Object Model)

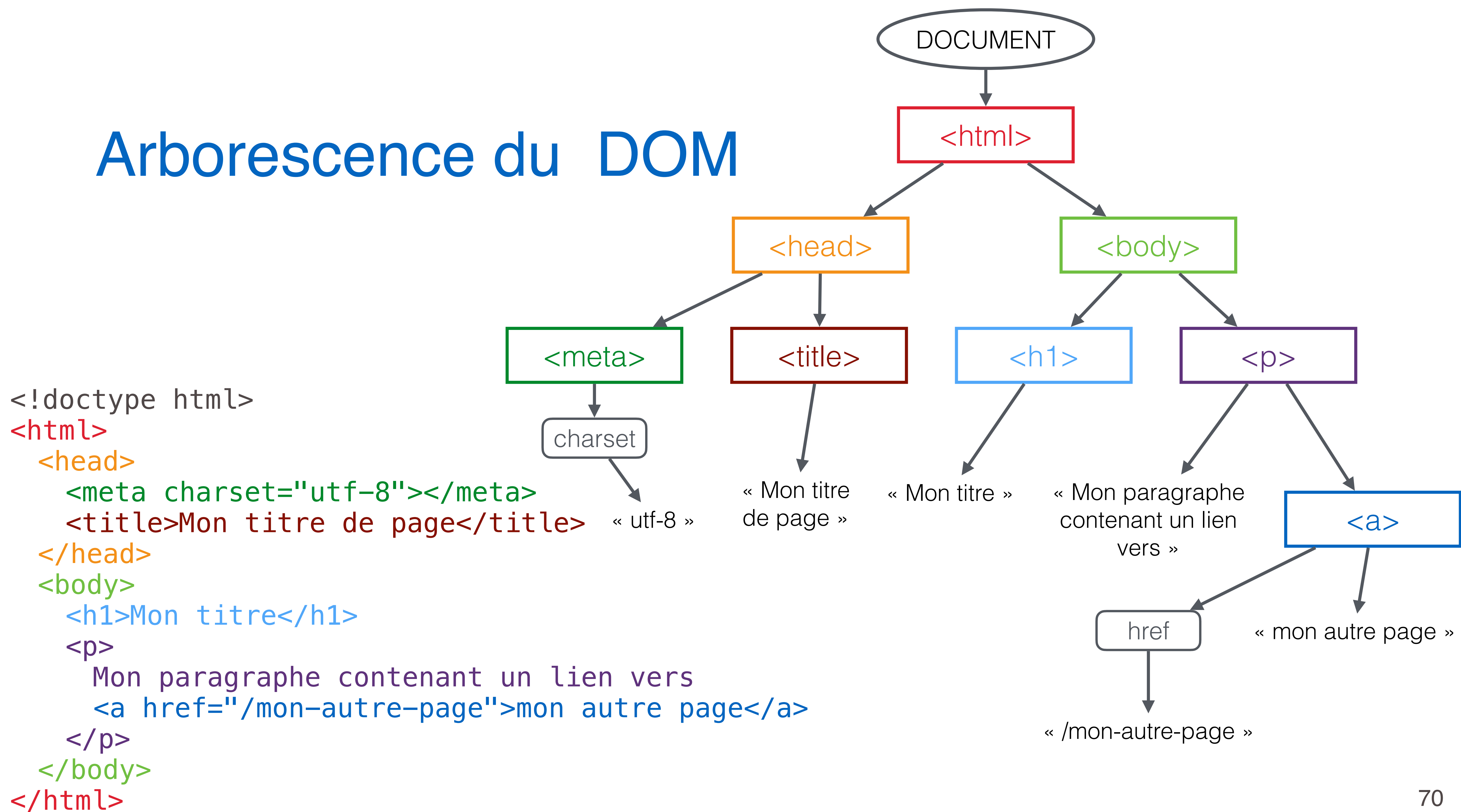
Représentation de la structure de la page web sous la forme d'une hiérarchie de noeuds.

Chaque balise HTML et chaque texte qui compose la page web y sont représentés par un noeud, sous la forme d'un objet.

Chaque objet/noeud dispose de propriétés et de méthodes permettant de le manipuler.

Avant 1998, chaque navigateur générait son propre DOM. Un standard a été défini cette même année par le W3C.

Arborescence du DOM



Accéder au DOM en Javascript

Bonne pratique : avant toute manipulation du DOM via Javascript, il faut que celui-ci ait été entièrement chargé par le navigateur afin d'être sûr de disposer de l'arbre DOM complet :

```
<script src="mon_script.js" defer></script>
```

Grâce à l'attribut defer, le script sera téléchargé en parallèle et exécuté seulement quand la page aura été chargée complètement.

NB : il existe plusieurs autres méthodes permettant de s'assurer que la page a été totalement chargée (`$(document).ready()` avec jQuery par exemple).

Accéder au DOM en Javascript

On accède à la **racine du DOM** avec l'objet **document** :

```
console.log(document);  
console.log(document.doctype);  
console.log(document.head);  
console.log(document.body);
```

Accéder aux noeuds enfants

```
console.log(document.body.firstChild); // premier enfant du body  
console.log(document.body.childNodes[0]); // premier enfant  
console.log(document.body.childNodes[1]); // deuxième enfant  
console.log(document.body.lastChild); // dernier enfant
```

```
document.body.childNodes[1].childNodes[3]; // imbrication
```

Accéder au noeud parent

```
var noeudEnfant = document.body.childNodes[0];  
console.log(noeudEnfant.parentNode);
```

Pour la suite de ce cours, nous conviendrons que la variable « `element` » utilisée dans chaque extrait de code correspond à un noeud dans l'arbre DOM : une balise « `div` » par exemple.

Sélection par Balise (Tag)

```
<!doctype html>
<html>
<head> [...] </head>
  <body>
    <div>
      <div>
        <p>toto<p>
        <p>tata<p>
      </div>
      <p>tutu<p>
    </div>
  </body>
</html>
```

Peut être appliquée à n'importe quel Element.

```
element.getElementsByTagName("p");
// tous paragraphes dans le sous-arbre (array)
```

3 éléments

NB : Quand une fonction dont le nom commence par getElement retourne plusieurs éléments (sélection par tag, classe... mais pas par id), le mot Element prend un « s ».

Sélection par Classe

```
<!doctype html>
<html>
  <head> [...] </head>
  <body>
    <div class="maClasse">
      <div>
        <p>toto<p>
        <p class="maClasse">tata<p>
      </div>
      <p class="maClasse">tutu<p>
    </div>
  </body>
</html>
```

Peut être appliquée à n'importe quel Element.

```
element.getElementsByClassName("maClasse");
// tous les éléments ayant la classe
// "maClasse" (array)
```

3 éléments

Sélection par Identifiant

```
<!doctype html>
<html>
  <head> [...] </head>
  <body>
    <div id="monId">
      <div>
        <p>toto</p>
        <p>tata</p>
      </div>
      <p>tutu</p>
    </div>
  </body>
</html>
```

Ne peut être appliquée que à Document.

```
document.getElementById("monId");
// l'unique élément ayant l'id "monId" (élément)
```

1 élément

ATTENTION : La fonction **getElementById** n'existe que dans **document**. En effet, les **id** étant uniques dans une page HTML, cette sélection se fait toujours depuis le document entier. C'est d'ailleurs pour ça que dans le nom de la fonction, **Element** ne prend pas de « s », les id sont unique.

Sélection comme en CSS (1/2)

```
<!doctype html>
<html>
  <head> [...] </head>
  <body>
    <div id="monId">
      <div>
        <p class="maClasse">toto<p>
        <p class="maClasse">tata<p>
      </div>
    <p class="maClasse">tutu<p>
  </div>
</body>
</html>
```

Peut être appliquée à n'importe quel Element.

```
element.querySelectorAll("#monId .maClasse");
// tous les éléments correspondants
```

2 éléments

Sélection comme en CSS (2/2)

```
<!doctype html>
<html>
  <head> [...] </head>
  <body>
    <div id="monId">
      <div>
        <p class="maClasse">toto<p>
        <p class="maClasse">tata<p>
      </div>
      <p class="maClasse">tutu<p>
    </div>
  </body>
</html>
```

Peut être appliquée à n'importe quel Element.

```
element.querySelector("#monId .maClasse");
// le premier élément correspondant (élément)
```

1 élément

Propriétés des noeuds/éléments

`element.innerHTML`

Définit ou obtient tout le balisage et le contenu dans l'élément

`noeud.textContent`

Définit ou obtient le contenu textuel d'un noeud et de ses descendants

`element.id,`
`element.href,`
`element.value, ...`

Définit ou obtient la valeur d'un attribut d'un élément.

```
element.textContent = "Hello World!";
```

<https://developer.mozilla.org/fr/docs/Web/API/Node>

<https://developer.mozilla.org/fr/docs/Web/API/Element>

Manipulation des attributs

Peuvent être appliquées à n'importe quel [Element](#).

```
element.getAttribute("href");  
// valeur de l'attribut href de l'élément
```

```
element.setAttribute("href", "https://www.google.com");  
// vérifie l'existence de l'attribut href de l'élément
```

```
element.hasAttribute("href");  
// vérifie l'existence de l'attribut href de l'élément
```

Manipulation des classes

Peuvent être appliquées à n'importe quel [Element](#).

```
element.classList;  
// tableau contenant la liste des classes de l'élément
```

```
if(element.classList.contains("maClasse")) { ... }  
// vérifie l'existence de l'attribut href de l'élément
```

```
element.classList.add("maClasse");  
// ajoute la classe maClasse à l'élément  
element.classList.remove("maClasse");  
// supprime la classe maClasse de l'élément
```

Création d'éléments dans le DOM

On commence par créer un élément (noeud), sans l'ajouter dans l'arbre DOM :

```
var newElem = document.createElement("a");
```

Nous lui ajoutons ensuite un contenu, puis un attribut href par exemple :

```
newElem.textContent = "Cliquez ici";  
newElem.href = "https://www.google.com";
```

Une fois sa configuration terminée (ou éventuellement avant aussi), nous pouvons l'ajouter au DOM :

```
document.getElementById("menu").appendChild(newElem);
```

L'élément est ajouté à la fin de (dans) la balise avec l'id #menu.

Création d'éléments dans le DOM

Ne peuvent être appliquées qu'à [Document](#).

```
document.createElement("p");  
// Crée un élément, sans l'ajouter au DOM
```

```
document.createTextNode("Hello World!");  
// Crée un noeud Texte, sans l'ajouter au DOM
```

Insertion d'un noeud dans le DOM

```
element.appendChild(nouveauNoeud); // ajoute le noeud dedans, à la fin
```

```
element.insertBefore(nouveauNoeud, elementReferentPourInsertion);  
// ajoute le noeud dedans, juste avant elementReferentPourInsertion
```

Remplacement/suppression d'un noeud

```
element.replaceChild(nouveauNoeud, noeudARemplacer);  
// Remplace noeudARemplacer par nouveauNoeud
```

```
element.removeChild(noeudASupprimer);  
// supprime le noeud
```

Modification des styles CSS

Les styles CSS d'un élément sont accessibles via sa propriété "style" :

```
element.style.color = "green";  
element.style.backgroundColor = "blue";
```

Événements

Le javascript permet de définir des actions à exécuter lorsqu'un événement se produit sur la page web.

```
element.addEventListener("click", callback);  
// Ajoute un Listener de clic sur l'élément
```

La fonction de callback est appelée lorsqu'un clic est fait sur l'élément.

Lors de son appel, l'événement est passé en paramètre afin de pouvoir être traité dans le callback (par exemple pour annuler le comportement par défaut (preventDefault())).

Événements

```
element.addEventListener("click", function (event) {  
    // event contient les informations sur l'événement (le clic)  
  
    event.preventDefault(); // désactive le fonctionnement par  
    défaut de l'événement (changement de page pour un lien par  
    exemple)  
  
    event.stopPropagation(); // empêche la propagation aux  
    éléments parents lors de l'imbrication de Listeners (si un  
    parent à un listener de clic par exemple)  
});
```

Performances

De manière générale, toute modification du DOM est gourmande.

Les derniers Frameworks Javascript (React...) utilisent des DOMs virtuels afin d'optimiser les performances.

Les Event Listeners sont également très gourmands en ressources. Par exemple, sur une liste, il vaut mieux n'en ajouter qu'un seul sur le `<ul>` qu'un sur chaque membre `<li>` de la liste. On peut alors, via l'argument du callback récupérer précisément sur quel élément `<li>` l'utilisateur a cliqué.

A yellow square containing the letters 'JS' in a bold, black, sans-serif font.

JS

Libraries vs Frameworks

Librairies vs Frameworks

Librairie

Offre un ensemble de fonctions et de méthodes que l'on peut réutiliser quand on le souhaite.

Nous appelons les fonctions et les méthodes des librairies quand nous en avons besoin.

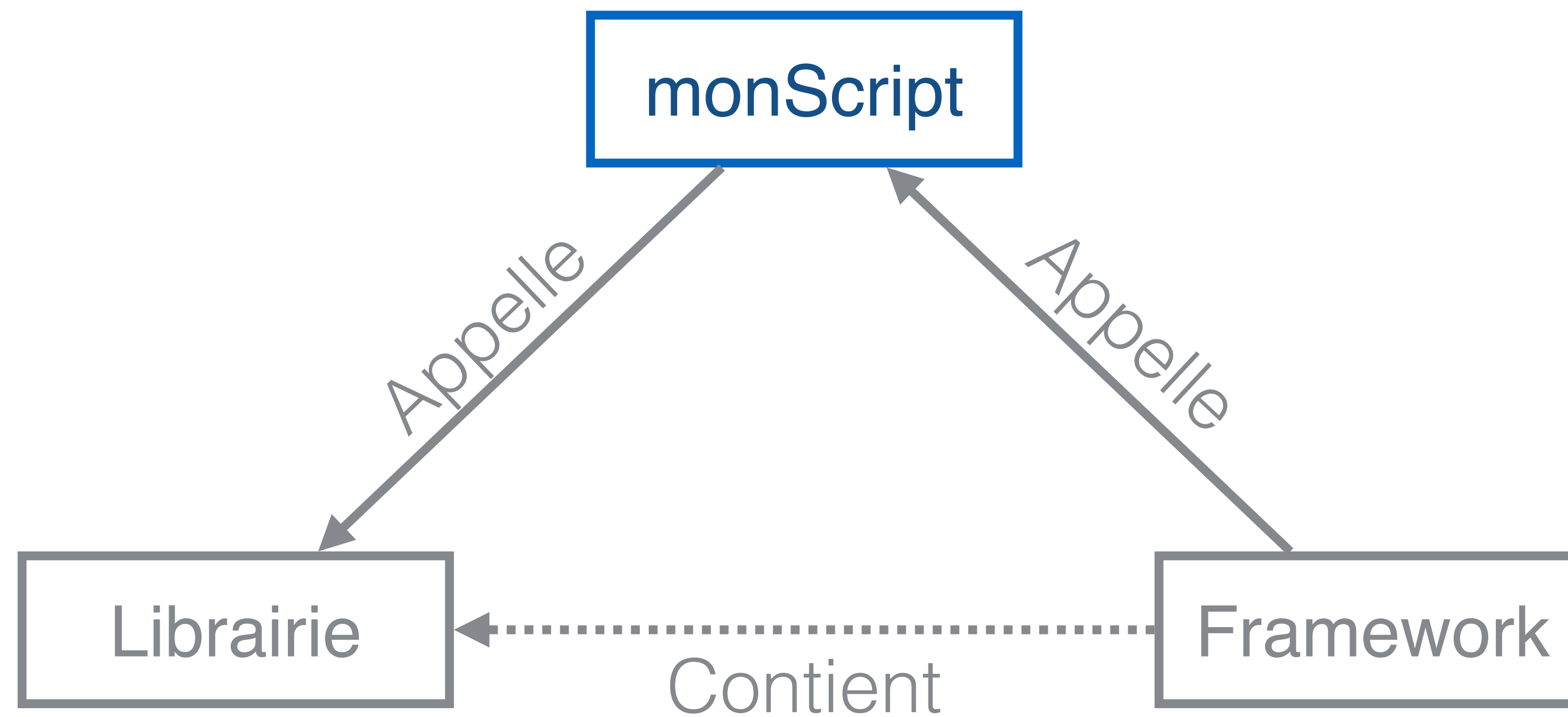
Framework

Définit un squelette dans lequel notre programme vient ajouter ses fonctionnalités.

Quand nécessaire, le framework appelle les fonctions et les méthodes que nous avons créées pour lui dans notre programme.

Plus complexe à prendre en mains.

Librairies vs Frameworks



Quelques librairies Javascript



jQuery :
Manipulation du DOM



Lodash, Underscore.js :
Manipulation de données

Quelques frameworks



AngularJS (v1), Angular (v2), Vue.js, React.js, Backbone.js... :
Frameworks permettant la création de **Single Page Application** (SPA)

JS

Exemples de frameworks PHP :





JS

Présentation d'une librairie JS :



Documentation



<http://api.jquery.com/>

Utilisation d'une librairie Javascript



La librairie doit être importée pour être utilisée :

// Bonne pratique : importer les scripts Javascript juste avant la fermeture du head ou du body:

<script type="text/javascript" src="library.js"></script><!-- jQuery par exemple -->

<script type="text/javascript" src="monScript.js"></script><!-- nos scripts en dernier -->

</body>

</html>



Possibilité d'utiliser un CDN (Content Delivery Network) :

<https://code.jquery.com>

✓ cache

✓ pas besoin d'héberger la librairie

✗ dépendance au bon fonctionnement du CDN

Utilisation de jQuery

jQuery étant une librairie permettant notamment de manipuler le DOM d'une page web, il est indispensable que cette dernière ait été entièrement chargée avant d'être manipulée.



La fonction **ready** exécute la fonction passée en callback quand la page entière a bien été chargée :

```
$(document).ready(function () {  
    // code ici. Le DOM, les images, les iframes... ont bien été chargés  
});
```

<http://learn.jquery.com/using-jquery-core/document-ready/>

<http://api.jquery.com/ready/>

Utilisation de jQuery

jQuery expose ses méthodes et propriétés via deux propriétés dans l'objet window : **jQuery** et **\$**. Ce dernier est un simple alias de **jQuery**, permettant une écriture plus rapide.

```
$(document).ready(function () { ... });
```

est équivalent à :

```
jQuery(document).ready(function () { ... });
```

NB : il est possible de stocker un élément jQuery dans une variable : **var maVar = \$("p");**

Utilisation de jQuery

```
$(document) // sélection du document
```

```
$("a") // sélection des liens de la page
```

```
$("body div a.maClasse") // sélection en notation "CSS"
```

```
$("body div").children("a.maClasse") // utilisation de fonctions
```

(EN) How jQuery Works?

\$(this)

Dans un callback, l'élément sélectionné est accessible avec **\$(this)**.

```
$(document).ready(function () { // le DOM est prêt

    $("a").click(function (event) { // quand on clique sur un lien ("a")

        event.preventDefault(); // on annule le comportement par défaut du clic
        alert("Quelqu'un a cliqué sur un lien !"); // on affiche un message

        $(this).hide("slow"); // On cache l'élément : il disparaît lentement

    });

});
```

Quelques liens vers la documentation

[Insertion à l'extérieur d'un élément du DOM](#)

[Insertion à l'intérieur d'un élément du DOM](#)

[Attributs généraux](#)

[Formulaires](#)

[Ajax](#)

...